

Application-level programmable internetwork environment

I W Marshall, J Crowcroft, M Fry, A Ghosh, D Hutchison, D J Parish,
I W Phillips, N G Pryce, M S Sloman, D Waddington

Future requirements for a broadband multimedia network are likely to be very different from those of today. Three key changes are identified — rapid introduction of new services, dynamic customisation of services by clients, and minimal management overhead. Application-layer active networking, perhaps the most pragmatic and immediately realisable active network proposal, is a potential solution to all three. In the ‘Management of Multiservice Networks’ project, BT and its academic partners are developing key middleware components required to support application-layer active networking.

1. Introduction

The characteristics and behaviour of future network traffic will be different from the patterns observed today, generating new requirements for network operators. Voice traffic will become another form of data, most users will be mobile, the amount of traffic generated by machines will exceed that produced by humans, and the data traffic will be dominated by multimedia content. In the medium term, the predominant multimedia network application will probably be based around electronic commerce capabilities. Operators will therefore need to provide a low-cost service, which offers an advanced global trading environment for buyers and sellers of any commodity. The e-trading environment will be equipped with all the instruments to support the provision of a trusted trading space. Most important is the ability to support secure transactions over both fixed and mobile networks. Networks will thus need to be robust, contain built-in security features and be sufficiently flexible to address rapidly evolving demands as other unforeseen applications become predominant.

Existing networks are very expensive, and the deployment of new communications services is currently restricted by slow standardisation, the difficulties of integrating systems based on new technology with existing systems, and the overall system complexity. The biggest cost is management. The network of the future will need to be kept as simple as possible by using as few elements as possible, removing duplication of management overheads, minimising signalling, and moving towards a hands-off network.

The simplest (and cheapest) current networks are multiservice networks based on powerful asynchronous transfer mode (ATM) or internetwork protocol (IP)

switches. New transport networks are designed on the basis that nearly all applications will eventually use Internet-like connectionless protocols. The difficulties of adding services such as multicast and quality of service (QoS) to the current Internet demonstrate that even these simpler IP-based networks will require additional mechanisms to enhance service flexibility. The simple transport network will therefore need a flexible service surround. The service surround will provide a trusted environment, with security features (for users, applications and hosts), QoS support, application-specific routing, automatic registration, and upgrade for devices connected to the transport network. It will also enable network computing facilities, such as secure gateways, application layer routers, cache/storage facilities, transcoders, transaction monitors and message queues, directories, and profile and policy handlers. Such a service surround is likely to be based on some form of distributed middleware, enabling the features to be modular and interoperable. The service surround must enable rapid introduction of new features by the operator. In order to minimise the management overhead, clients will directly control which features should be used for a particular session, without operator intervention. As a result, the network will need to know nothing of the semantics of the session. To achieve this a middleware based on some form of active services or active networks will be required. Previous attempts to achieve flexibility based on CORBA [1] suffered from the difficulties discussed in McKee et al [2].

One of the great strengths of the Internet is that a number of its functions are loosely coupled (naming, addressing, routing, congestion and flow control, to name but a few). This means that partial failures of the network do not bring the whole system to a grinding halt. It has its

downsides, but appears to be a good basis for doing things. BT has funded a programme called the 'Management of Multiservice Networks' (MMN) University Research Initiative since 1994 which is exploring how this design philosophy is reflected in a programmed environment. The programme has developed a number of key active network middleware components, including an active server, resource controllers and monitors, and management policies. This paper introduces active networks and justifies the choice of application-layer active networking. It then describes the components developed to date by the academic partners, and discusses the ongoing research that is required.

2. Active networks

Active networking was originally a proposal, by Tennenhouse at MIT, to increase network flexibility by adding programmes to the packet header, that are intended to run on network devices that the packet encounters [3]. This is referred to as the capsule approach. There are a number of problems.

- The maximum transport unit (MTU) size in the Internet is typically 576 bytes. This will likely be upgraded to 1500 bytes in the near future; however, it is clear that if there is to be a programme embedded in every packet the programmes must be very small, even if the programme is not confined to the header. This severely restricts the flexibility that can be offered, although it has been shown that copy instructions to emulate multicast can be embedded in packet headers in some circumstances.
- It has been proposed that only those packets initiating flows should carry programmes. However, it is common for the packets in an individual flow (such as a document retrieval) to use multiple routes across the Internet. This is a result of the routers having the freedom to use the best available route at any time, so as to maximise network resiliency. Therefore, in order for a programme to be applied to all packets in a flow, either the route for all subsequent packets in the flow must be pinned, so that all packets flow through the node where the programme was loaded, or, the programme must be copied to all nodes on valid routes. The second option is clearly impractical. The first option is currently not possible, and in any case creates an undesirable reduction in network resilience.
- The proposal envisages programmes being supplied by network clients. However, service operators will never permit third-party programmes to run on their equipment without a strong guarantee that the programme will not degrade performance for other users. Such a guarantee requires the programmes to be written in a language in which behaviour is verifiable through pre-run checks, resource usage can be tightly

controlled and termination is guaranteed. The Safetynet [4] project at Sussex University in the UK is designing a promising language, but the research is still at a very early stage and designing programmes which safely invoke other programmes is not currently possible. Since it will be extremely hard to create interesting programmes in a language which is simple enough to enable complete resource control and termination guarantees, the flexibility offered by this approach is probably somewhat limited, even when the language is mature.

- The programmes are intended to be added to the switch-control kernel in the router. All the known approaches to making the kernel extensible degrade performance. Packets which do not require router programming will thus suffer an unacceptable performance penalty.
- Undesirable interactions between programmes and network features are almost impossible to predict and control. For example, a mobile client will potentially send programmes to several routers where they are run. However, the routers do not receive the acknowledgement packets that would terminate the programmes as the 'acks' are routed elsewhere after the client's location has changed.
- Standards for the interface offered by active routers must be developed before any service based on this proposal could be offered. Appropriate standards are not even being discussed at present.

Despite the manifest difficulties inherent in this proposal it has succeeded in highlighting an important requirement, and stimulating discussion among a previously disparate community of researchers attempting to develop more immediately realisable means to resolve the requirement. The main threads are summarised in the next section.

3. Active networks and services

The first response to Tennenhouse was a somewhat different flavour of active networking, in which the packets do not carry programmes but transport layer header flags indicating the desirability of running a programme [5]. This approach attempts to resolve the issue of restricted programme size, and potentially gives network operators the freedom to choose an appropriate programme of their own which has been tested. However, the proposal makes no progress on the last three issues, and the range of flags could not be large as the space available in the transport layer header is tiny. This proposal has impacted the Internet Engineering Technical Forum's (IETF's) 'diffserv' activity which is enabling QoS in IP networks by adding flags to transport headers, one of which is an active tag.

The second response came from the programmable network community who had for some time been looking to make networks more active with respect to operators [6] and had progressed to a concept called 'switchlets' [7] in order to avoid requiring operator intervention for all programmable changes. Switchlets enable clients to control their own VPNs by downloading their own control software on to a designated subset of the switch. Switchlet development at Cambridge has been partly funded by the MMN project, but is not covered in this paper since it only provides flexibility within a VPN for a single large customer and is not currently a significant part of the design for an active services infrastructure. Programmable interfaces are being standardised in IEEE P1520 [8].

Alexander and co-workers at the University of Pennsylvania and Bellcore are working on a proposal (switchware [7]) which combines programmable packets, switchlets and a safe language (PLAN). This could be regarded as a realisable version of Tennenhouse, but only in the long term.

A third alternative, developed by UTS and BT, and known as application-layer active networking (ALAN) [10], is perhaps the most immediately realisable. Similar proposals [11, 12] were described as active services. In these systems the network is populated with active nodes referred to as service nodes, or dynamic proxy servers. These can be thought of as equivalent to the hypertext transfer protocol (HTTP) caches currently deployed around the Internet, but with a hugely increased and more dynamic set of capabilities. Logically they are end-systems rather than network devices. This approach relies on redirecting selected packets into an application-layer protocol handler, where user space programmes can be run to modify the content, or the communication mechanisms. Packets can be redirected using a single active packet tag in the transport layer header, or on the basis of the mime-type in the application-layer header. There is no need for additional flags or for any new standards (indeed many implementations use the ubiquitous HTTP, and an arbitrarily large number of programmes, of arbitrary size can be used. The programmes can be selected from a trusted data source (which may itself be a cache) containing only well-tested or specified programmes, and can be run without impacting on router performance or requiring operator intervention since they do not have any impact upon the control kernel for normal packets. As programs can be chosen to match the mime-type of the content (in the application layer header), again no additional data or standards are required. Alternatively a more detailed specification can be supplied in XML metadata [13], if desired. There is a small performance penalty associated with the redirect operation, but this is acceptable for most applications. The major outstanding issue is the interaction between dynamically loaded programmes, and this should be a priority for ongoing research.

There is a further proposal [14] that allows servers to supply cache applets attached to documents, and requires proxies to invoke the cache applets. Although this provides a great deal of flexibility, it lacks important features such as a knowledge-sharing system among the nodes of the network (it only allows interaction between the applets placed in the same page). The functionality is also severely restricted by the limited tags available in hypertext markup language (HTML). Most importantly the applets must be supplied by the content source and cannot necessarily be trusted. Clients do not have the option of invoking applets from trusted third-party servers. Using mime-types (as in ALAN) provides more flexibility than HTML tags, but still restricts the range of applications that can be specified by content providers, because different operations are often required for content with identical mime-types. It is therefore necessary to find a better way to specify new services. XML provides a very promising solution, since the tags are extensible and authors can embed many different types of object and entity inside a single XML object. For example, policies describing resource and security requirements can be expressed and transferred with the object.

4. ALAN

The design is built in several layers and is based on existing technology. Figure 1 shows the architecture of the prototype. The first layer is a fully populated cache hierarchy, with caches placed at all domain boundaries and network bottle-necks. It is envisaged that active nodes and caches be collocated since the optimal sites for most of the activities proposed for active networks are domain boundaries. In addition, it is advantageous to maintain a cache of programmes required by the active services at an active node and a Web cache is a convenient implementation. For the prototype, Squid v1.19 [15] was used for the cache. The second layer and the upper layers constitute the core of the system and will be discussed in detail within this paper. An ALAN platform implements the active services. One of these services is an XML parser that provides the functionality to handle metadata associated with objects.

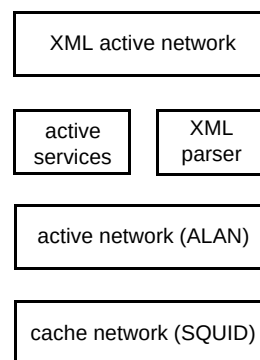


Fig 1 XML+ALAN platform.

The ALAN platform is a Java Remote Method Invocation (RMI) based system designed to host active services. It provides a host program known as the dynamic proxy server (DPS) that will dynamically load other classes (proxylets) that are defined with the following interface — load, start, modify, stop [10].

The platform provides a proxylet that analyses the HTTP headers and extracts the mime-types of the objects passing through the machine (HTTP parser). After determining the mime-type of the object, the program chooses a content handler, downloads the appropriate proxylet from a trusted host to handle that mime-type, and starts the proxylet with several parameters extracted by the HTTP parser. However, there is a need for additional data (not included in the HTTP headers) to manage interoperability between the services and to expand the flexibility and range of applications that can be developed. XML provides a mechanism to implement these improvements and appears a perfect complement to the architecture. A simple XML parser built in Java works in collaboration with the HTTP parser, and is designed to utilise all the metadata needed in the active applications.

The functionality of an active node (Fig 2) is described as follows. Upon the arrival of an object into the node, the HTTP parser examines the header and gets its corresponding mime-type. If the object is an XML object, then the XML parser is called and it will extract the meta-data. The metadata specifies which proxylets should be invoked, in which order, with which parameters, and under what circumstances. The parser then makes the appropriate calls to the DPS, which loads the proxylets.

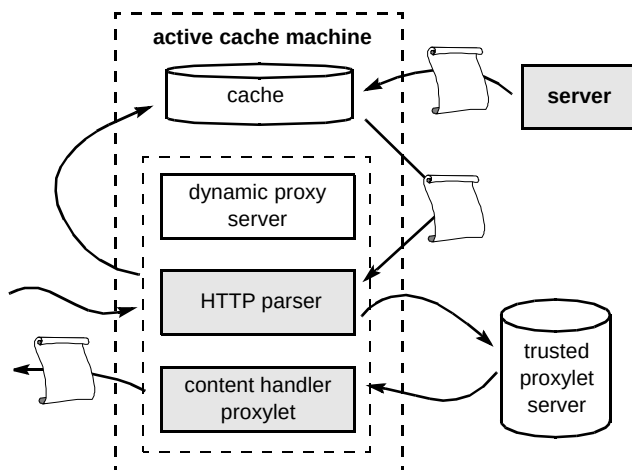


Fig 2 Functionality of an active node.

A key requirement of distributed application-level services is the selection of the appropriate server from a pool in a scalable way. A proximity metric can be delay, or throughput, and may reflect both network and application-level performance. In a world where there are many DPSs

distributed around the world, a mechanism is required to discover the optimally located DPSs for a particular task. A mechanism analogous to a routing protocol will need to be adopted to allow the discovery of a DPS. In this work, the proximity service would be built as a unification of two separate services, one host-based and the other router-based. The router-based service would be created using ideas from the Internet routing protocols OSPF (open shortest path first) and BGP (border gateway protocol) (e.g. link state adverts from OSPF). This problem can be solved by relating DPS location to the caching hierarchy, and using application-layer host-based routing techniques developed by the caching community — thus still operating at the application level, and not touching the underlying transmission system, which may be IP, ATM or whatever is available. The aim is real-time construction of overlays that are self-organising to meet the needs of the application and its instantaneous environment.

DPS discovery will need to be performed based on a number of metrics including:

- allowed domains,
- closest host (in terms of bandwidth or latency),
- some fault-tolerance metrics,
- processor availability (to support heavily processor-intensive transcodings).

One of the perceived problems with allowing arbitrary proxylets to run on DPSs is the possibility of undesirable interactions. Rather than attempting to exhaustively test a proxylet, it may be sufficient to force it to live within tightly managed resource constraints so that it cannot affect other proxylets run by other customers. It is then the responsibility of the user or author of a proxylet to verify that a particular proxylet behaves correctly. Of course it should still be perfectly reasonable for a proxylet to run for extended periods of time in some contexts. A mechanism could be developed to accept some form of electronic cash in order to continue to run a proxylet. Owners of a DPS will not mind that a local resource is being consumed if they are being suitably recompensed.

In order to address the management problems the project is now attempting to build mechanisms for proxylet admission control and policing, based on the work of all the partners. This involves the definition of proxylet metadata, and permits DPS load balancing. A policy-based QoS management via feedback monitoring to an adaptive streaming proxylet is also being developed.

5. Dynamic stacks and policies

5.1 Dynamic stacks

Imperial College has built a flexible distributed programming environment (DPE), Regent, that complements the work on application-level active networks performed at UTS [10], by enabling the protocol stack to be configured to match the QoS requirements of each proxylet using it. Regent differs from existing DPEs, such as CORBA ORBs [16] or Microsoft's DCOM [17], in two respects — multiple interaction styles between application components and dynamic protocol stacks.

Application components are not limited to object-based request/reply interactions. A language has been developed (Midas [18]), which permits the programmer to write an interaction specification in terms of asynchronous messages and state machines. This allows the succinct specification of the various interaction styles required by a large distributed system, such as request/reply, event dissemination, bulk data transfer or media streams. Midas syntax is similar to the CORBA Interface Definition Language (IDL). It uses IDL syntax for constant, type and module definitions and replaces object interface definitions with definitions of interactions following the model described above. Midas also allows the definition of generic types and interaction styles that are themselves parameterised by type, and so does not need an 'any' type to represent untyped data.

Midas specifications are compiled into implementation language constructs, such as Java classes, that define the message interfaces and provide distribution transparency via proxy objects. Midas allows interactions to be defined independently of the implementation language and simplifies the task of the programmer by generating support for distribution transparency.

The run-time support, generated from Midas specifications, interfaces with the Regent transport sub-system. This sub-system provides an object-oriented framework for implementing transport protocols. The framework takes a component-based approach — lightweight protocol layers encapsulate small aspects of protocol functionality and are composed into protocol 'stacks' (actually directed graphs) in order to implement the required transport protocol.

Protocol stacks (see Fig 3) can be identified by composite names that specify the layers within the stack. These names can be 'elaborated' by the Regent run time to dynamically load implementations of the named layers into the node, instantiate layers, and compose them into the stack described. References to distributed services contain these stack names, allowing servers and clients to evolve

independently, making use of new transport protocols as they become available.

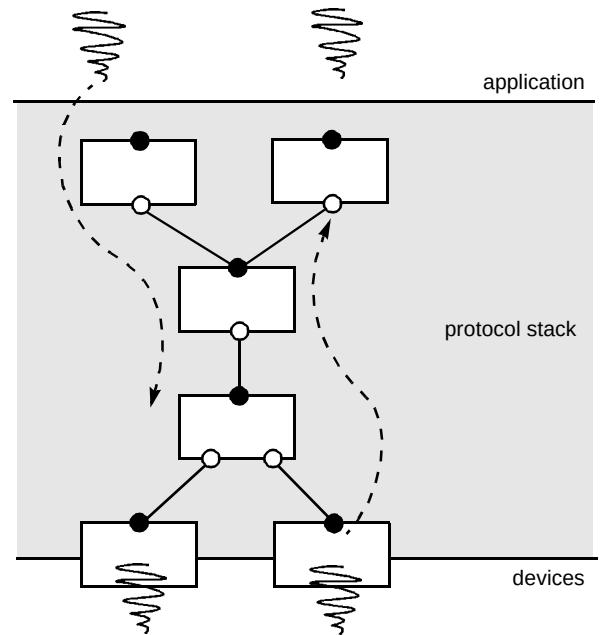


Fig 3 Protocol stacks.

A protocol layer can also provide control interfaces through which higher layers can control the operation of the protocol. Control interfaces can also expose events to higher layers. To be notified of events occurring beneath it in the stack, a protocol layer must implement a compatible 'event listener' interface and register the interface to be called back when events occur. The model of attributes and events used by control interfaces is similar to that used by the Java Beans component framework.

A useful type of protocol layer is the 'virtual protocol', that does not modify the packets passing through it. Virtual protocols can be added to a stack without affecting its compatibility with any remote system communications. Virtual protocols that expose control interfaces and events are the basis for configuring monitoring and management support into a node's communication sub-system.

5.2 Policies

Imperial College has also developed a policy notation and support tools which permit the specification of 'authorisation' and 'obligation' policies [19–22]. Authorisation policies specify what activities a manager is permitted or forbidden to do to a set of target objects and are similar to security access-control policies. Obligation policies specify what activities a manager must or must not do to a set of target objects and define the duties of a manager. Obligation policies define actions that are triggered by events. As policies are interpreted by

automated manager agents, the behaviour of the agents can be modified dynamically by changing policy rather than re-coding. The policies thus provide a constrained form of 'programming' of automated agents to change management strategy without shutting down the management system.

The Darwin Architecture Description Language (ADL) is being used to specify the application-level programmable inter-network environment (ALPINE) components required for a particular application. This will give a declarative specification of what proxylets are required, on to which dynamic proxy servers they should be loaded, what protocol stacks are potentially required, and how they should be configured. It is envisaged that Darwin statements will be embedded in the XML metadata. The Darwin language allows a component's implementation itself to be distributed. This would allow the specification of proxylets that span multiple DPS servers, making best use of available communication, computation and storage resources.

The advantage of an approach based on the use of Darwin is that more checking can be performed at design time to detect potential incompatibilities and problems. Of particular benefit is the fact that architectural descriptions are an ideal skeletal framework for other support tools, e.g. graphical design tools, compilers, behavioural and performance modellers, debuggers and application management software. The Midas language integrates with Darwin to allow the modelling and analysis of designs that take the component behaviours, interaction protocols, and transport layers into account.

The proxylets required to perform a particular application and the protocols they use may need to adapt to events such as component failures or changes in the quality of service during a long-running interaction. The obligation policies provide a flexible means of specifying how the systems should adapt to these changes. The advantage of the obligation policies is that they are interpreted and so can be dynamically disabled/enabled or replaced without stopping the manager agents, i.e. the management policy of the active networking can be dynamically changed as well as the proxylets themselves. The research objective will be to see whether the event-based obligation policy rules can be used to control proxylets and protocol stacks.

As part of the investigation of policies to manage transport protocol stacks, it is intended that virtual protocols are inserted into a binding's stack to monitor the actual quality of service available to the binding. If the QoS varies beyond some predetermined threshold, control events can be generated. Management agents within the node can use these events to trigger policies as to how to react to changing QoS levels. Potential actions could include fine-tuning the operation of the protocol stack via the control interfaces exported by layers in the stack, replacement of the stack with another which makes better use of network

resources, or the triggering of application-level changes, such as changing the media encoding used or selection of a different dynamic proxy server from those available in the ALPINE framework.

Another issue addressed by the policy framework is security. Authorisation policies can be used to define what proxylets or stacks can be loaded by a server, who can install proxylets on a proxy server, etc. Obligation policies can be used to specify actions to be taken in the event of security violations.

6. Server resource controls

Lancaster University's work falls under the broad title of 'End-system QoS Management'. Management of distributed multimedia application sessions involves three distinct processes — QoS specification which captures application-level QoS requirements, QoS mapping which translates user-level requirements to finer grained network and operating system level requirements, and QoS assurance which is the maintenance of the end-to-end service through resource monitoring, policing and adaptation. This work has principally concentrated on providing suitable tools, abstractions and middleware mechanisms for the support of these processes in the end-system. Figure 4 illustrates aspects of the end-system that are directly addressed by this work (the shaded areas).

Many current distributed object environments, including CORBA [23] and DCOM [24], do not support suitable abstractions for continuous media interactions. In this architecture a binding object, which manages one or more bindings, takes references to one or more point-to-point requirements (a point being an interface and the requirements being stream typing and QoS) and maps these requirements on to a configuration of components. To aid the task of mapping, the concept of hierarchical binding has been introduced. Hierarchical binding means that a binding object 'delegates' the task of mapping from point-to-point requirements to further sub-bindings. This approach to binding means that each binding object need only retain enough resource mapping information to break down a point-to-point requirement to two or more finer grained requirements. For instance, an end-to-end service binding can be reduced to two bindings in the end-systems and one binding in the network. All services in this architecture are essentially provisioned through one or more hierarchical, distributed bindings between components (such as proxylets or user applications). Binding objects automate the selection, configuration and control of resources between two or more interfaces.

Once a service has been created, there is the problem of how to control, manage and dynamically adapt the service during its lifetime. The hierarchical binding tree is an ideal mechanism for the support of QoS monitoring and QoS

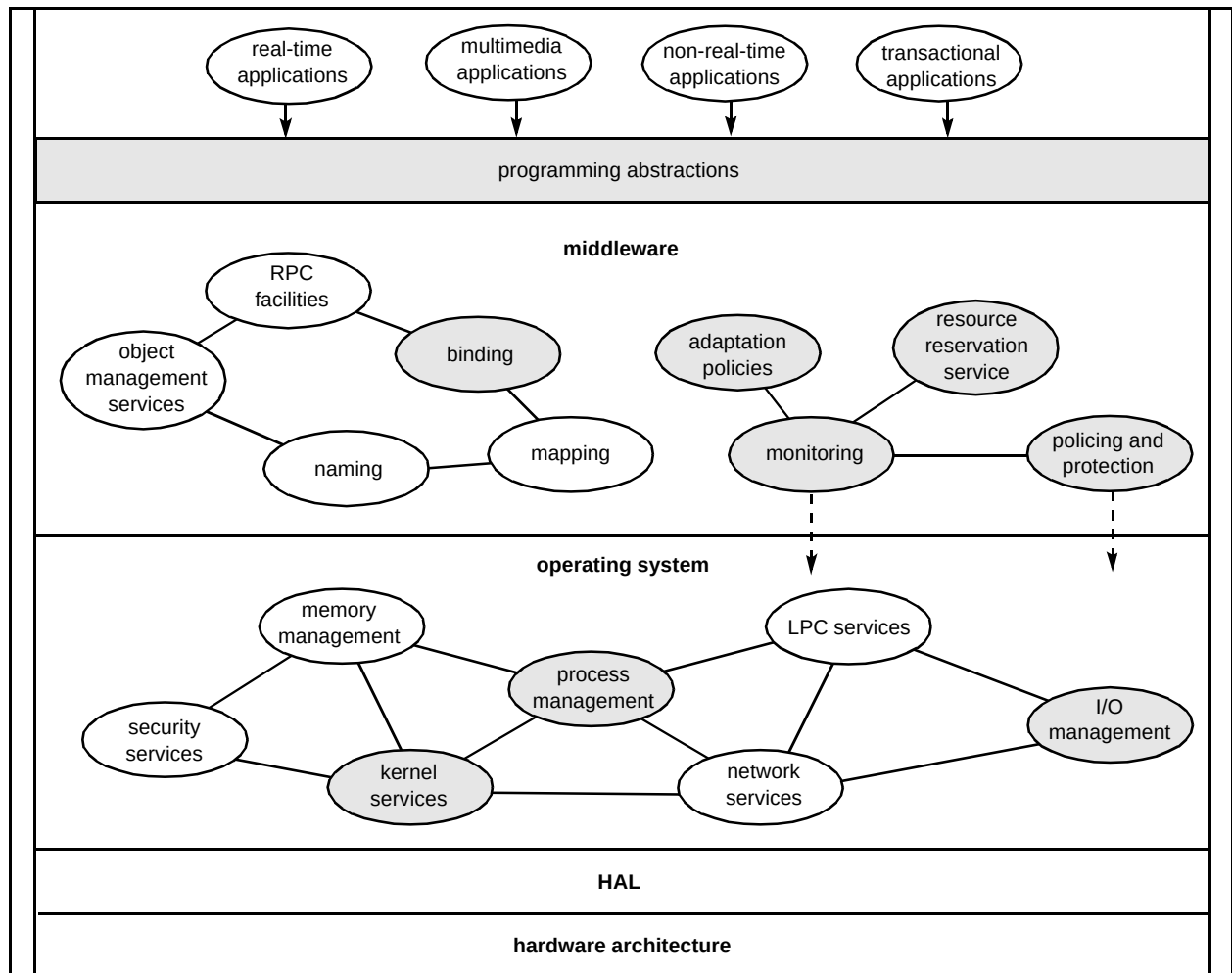


Fig 4 QoS management architecture.

management. Other researchers [25] have proposed the introduction of a single QoS manager component, where the QoS manager is responsible for QoS assurance through resource allocation, monitoring and maintenance in both the end-system and the network. However, such a component would require a huge amount of mapping and management knowledge to support large-scale distributed applications, and that service management through a single entity is too centralised and severely inflexible. The solution here is to build the QoS management into the bindings themselves, i.e. to incorporate the management functionality into the communication mechanisms between one or more interfaces. Expressing the requirements as policies in proxylet metadata, and using a dynamic protocol stack, enable the management to be associated with the binding in precisely this way. Each binding is responsible for a point-to-point contract and must make every effort to maintain this agreement. The binding may employ control invocations on lower level entities or even carry out component re-configuration. If the binding is unable to sustain an agreed contract, it should signal QoS failure or degradation to its parent binding. If the binding object is the service binding object, i.e. the root of the binding tree,

feedback is given directly to the client application. The client application may then decide on an alteration of service requirements. One of the issues of such an approach to the management of a service is that the component, which is responsible for the degradation of service, carries out the necessary action to resolve the problem instead of relying on other components within the session to take action.

In order to suitably control and co-ordinate the utilisation of end-system resources between multiple applications, services for resource reservation, admission, monitoring, policing and adaptation must be introduced. Mechanisms for these processes are provided by the end-system resource manager (ERM) which has been developed as a multimedia, middleware system service (Fig 5). A form of 'loose' resource reservation has been employed, which means that resources are reserved, admitted and released, as with standard approaches to resource reservation, except the system cannot offer firm guarantees on the availability of resources even if reservations have been made *a priori*. The system therefore also uses dynamic QoS management to respond to unforeseen variations in resource availability. The infrastructure offers a best-effort management service,

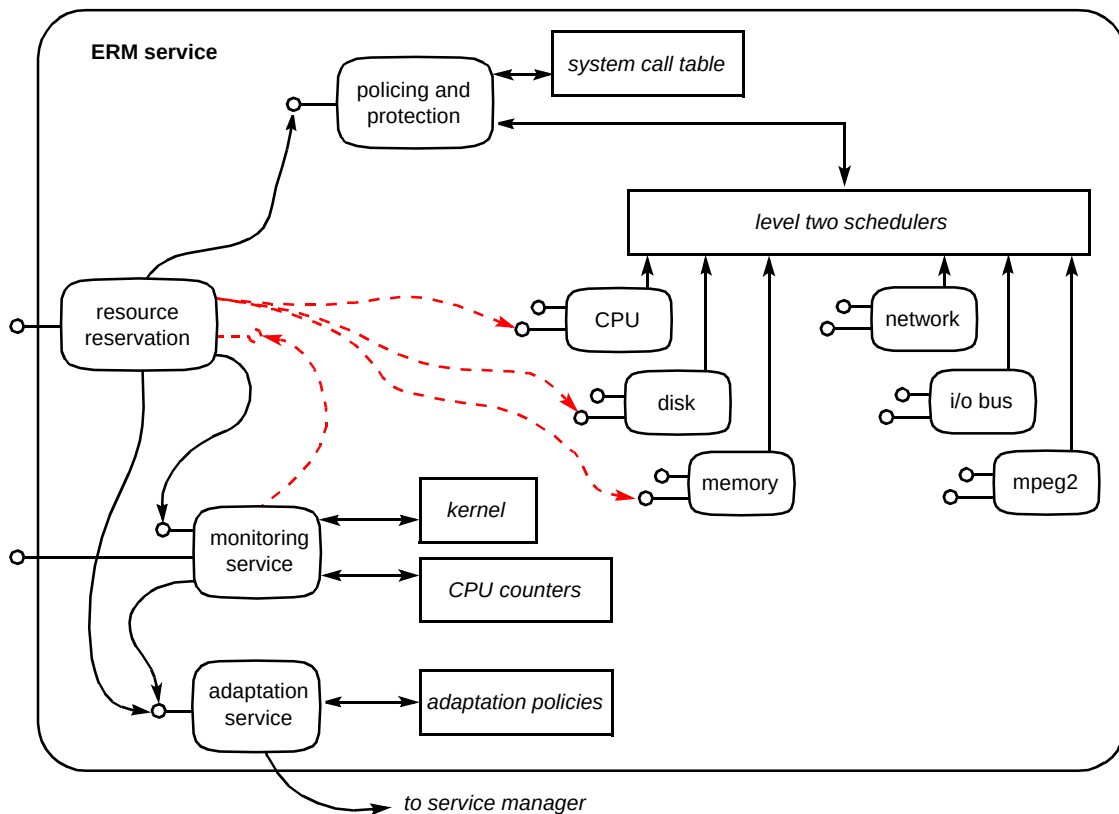


Fig 5 End-system resource manager.

and will take all possible measures, including the use of policing and adaptation, to ensure that reservations which have been admitted by the system are suitably respected. The main reason for the use of this approach is because of the relatively unpredictable availability of resources within a non-real-time priority-based operating system such as Windows NT. Furthermore, even if sufficient policing and guaranteed resource scheduling can be introduced, neither the legacy applications nor the operating system itself can be expected to follow a mandatory reservation process.

Each resource managed by the ERM is encapsulated as an extensible 'resource module'. These modules are responsible for the fine-grained allocation and scheduling of resources according to earlier reservations the client made via the ERM. Generally, each of the resource modules requires access to scheduling and control mechanisms at the lowest level, i.e. within the operating system executive. Mechanisms already prototyped include kernel modules for resource policing (interceptors for disk i/o and memory allocation, and specialised thread scheduling for processor management). The scheduling has also been arranged in a split-level scheme. The first level of scheduling fulfils the role of translating relatively coarse-grained requirements into requirements which can be released by the respective low level, or level 2, schedulers. This approach means that the complex task of translating QoS/resource abstractions into meaningful scheduling requirements is only carried out at the point of admission, thus avoiding any degradation in

general system performance. Furthermore, split-level scheduling means that application-specific reservation abstractions can be introduced without the need to adjust the underlying level 2 schedulers.

In conjunction with the resource reservation services provided by the ERM, the service also incorporates services for QoS monitoring. To do this the ERM acts as a retrieval agent for extensible performance counters in the operating system. These counters are generally associated with the resources in the end-system and the actions they perform. Furthermore, application-specific counters may also be introduced. This is particularly useful for monitoring activities in continuous media processing, such as missed frames or frozen buffers. The monitoring information can be retrieved by a remote client, via the ERM. This feedback can also be used by the run-time management mechanisms to automate responses to fluctuations in available resources or alterations in application level requirements. Within the work a general model of QoS adaptation [26] has been developed, which is used to co-ordinate the balance of resource utilisation across the end-systems and the network. Adaptation decisions are made in response to negative feedback from resource monitoring in the end-system, and may take the form of resource control or end-to-end reconfiguration. Currently adaptation policies are statically defined by the system administrator and cannot be adjusted dynamically — aspects of dynamic policy specification will be addressed in future work.

7. Network resource controls (UCL)

UCL is engaged on work on the management of protocol stacks within end-systems. Resource reservation protocol (RSVP) can ensure that time-critical applications receive the resources they need, but only at the cost of using a non-scalable amount of router CPU resource. Instead of controlling traffic within the network where CPU resources are scarce, the aim was to control how much traffic enters the network at its edges. The work is focused around transport protocol performance control through remote access from a management centre to the key performance parameters in a protocol. The system can operate in a client and, in principle, in a server, and can be added without rebuilding (or re-compiling) the application or operating system protocol stack. Remote access from Java RMI, CORBA remote procedure call (RPC) and simple network management protocol/common management information protocol (SNMP/CMIP) are all possible. The system is intended to use feedback from the Loughborough work which makes available detailed network performance statistics (see below). The basic idea is to instrument the TCP-IP stack so that any application can be rate controlled dynamically via network management. Instead of embedding congestion control algorithms in the TCP-IP stack itself, different rates can be applied to different applications depending on their QoS requirements.

A prototype of the protocol stack management system has been developed. The purpose of the system is to provide a standard network management interface to the protocol stacks running in Windows NT computers; a network management agent running on each computer monitors and controls the traffic running through an instrumented Winsock2 TCP-IP stack. Network managers can use the system to control transparently how much network bandwidth an Internet application can use — in this context, transparently means that the controls can be applied without any changes to the Internet application. Bandwidth controls can be applied to a group of hosts or a single host, to a group of processes within a host or a single process and to a group of sockets within a process or a single socket.

The protocol stack management system consists of a number of components, as depicted in Fig 6.

An instrumented Winsock2 TCP-IP dynamic link library (DLL) monitors and controls traffic at the lowest level of the system. The DLL reports how much traffic is being sent on each socket and allows a manager to block and rate control sockets. A Java agent running on each machine controls the Winsock2 TCP-IP stack. It presents a CORBA interface that allows network managers and applications to decrease and increase throughput rates for any socket. It uses a purpose-built reliable multicast protocol to send monitoring information; multiple managers and applications can retrieve monitoring information

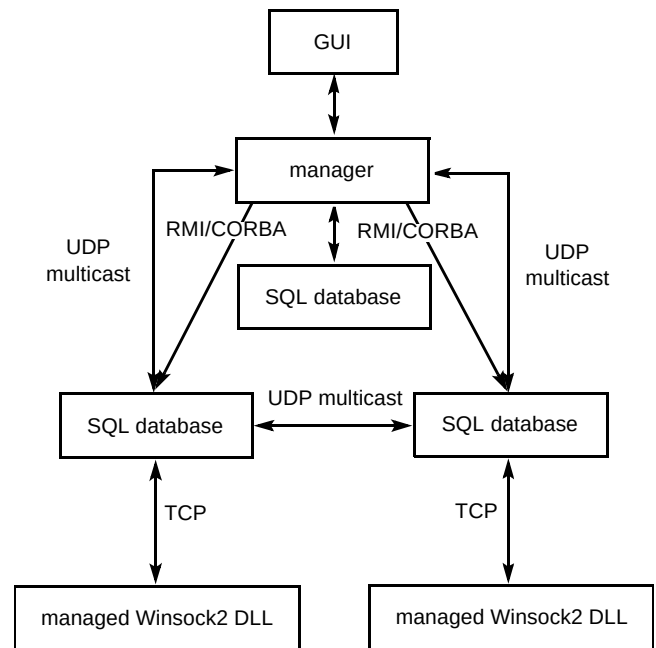


Fig 6 Stack management system.

without swamping the network with management traffic. A central manager with a graphical user interface (GUI) allows network managers to communicate with the agents on each computer and control how much traffic each host and process can inject into the network. A relational database system stores monitoring information gathered from each agent. It logs how much traffic each host, process and socket has generated.

Recently the focus of this research has been to deploy the protocol stack management and enhance it so that it can be used to do Internet server analysis and control. The system is being deployed on an array of servers at BT Laboratories (BTL). The monitoring capabilities of the system have been extended so that it can do traffic analysis on a per minute, per hour, per day or per month basis. Traffic statistics can also be presented on a user basis rather than just a host basis. The system could be extended to do Internet pricing which is likely to be an effective traffic control measure.

In the first instance the stack manager serves the purpose of limiting the amount of network resource available to proxylets, and preventing interference between different flows originating from proxylets. In the future the plan is to use it as the basis of a low-level implementation of the DPS that enables the addition of transport-layer services as flexibly as application-layer services. Current IP routers provide a very simple forwarding function, together with a complex route exchange and computation function. As further services are added, such as integrated and differentiated services, mobility support, security and so forth, the control functionality concerned with both the routing and the forwarding function becomes more complex. Proponents of 'strong active networks' claim that,

provided there are sufficient safeguards, distributed programs may be dynamically updated to create new services 'on the fly'. This area of work requires a parsimonious approach to adding programmability to routers. The high-level virtual machine (VM) used in ALAN will need to be re-engineered to operate at a lower level in the operating system, while still supporting the high-level proxylet interface. Based on the work at Sussex University [2], a low-level VM will be designed that would interact with the baseline (unicast, multicast, mobile, QoS) routing and forwarding functions, and allow combinations of low-level operations to make up higher level programs.

8. Internet monitoring

In order for the management and control mechanisms outlined above to operate correctly and fully support the DPS, it is necessary to have access to dynamic information about the state of the network at remote sites. The DPS is intended to establish proximity of nearby active nodes through delay-based measurements dominated by the response time of the remote server. However, to maintain QoS means that the DPS will need access to path-level metrics such as bandwidth and service classes (QoS) available on each path to nearby active nodes. It will also need information on current load (per QoS class), loss rate and burstiness for each path. Such information is hard to obtain in the current Internet. Loughborough University have been funded by the MMN URI to develop network performance measurement tools which could evolve towards satisfying these requirements. The work commenced with the development of a distributed architecture for network performance measurement which is generally applicable to any packet or cell-switched network. This architecture consists of a number of (relatively simple) remote stations whose task is to generate and receive test messages via the network. These messages are generated at the ultimate command of a control station whose complexity is greater than that of the remote stations and includes the ability to retrieve, store and process the measurement data. A suite of software tools associated with this station can produce various summaries of network performance for research or operations-based users.

An instance of this architecture [27] has been deployed on BT's Switched Multi-megabit Data Service (SMDS) network where IP packets are generated, received and analysed in order to help understand the performance of the network at higher levels of abstraction than merely that of basic connectivity. This system has now been running operationally in excess of two years.

The system architecture has also been enhanced using the object-oriented paradigm. The components (e.g. remote stations, databases) are now viewed as objects allowing greater flexibility in future deployment and development. In particular, it allows a less centralised model in which all active nodes can be control stations.

Considerable work has been conducted on enhancing this object-oriented architecture [28]. It is a fact of network performance analysis that the basic monitored data is often repeatedly processed in different ways as the analysis task develops. This becomes a considerable processing challenge to the system and its database, and is also inefficient as basic processing operations are often repeated. Work has led to the development of a database using intermediate information structures to allow subsequent reuse of the data to be efficiently and safely achieved when multiple user information requests are processed.

The architecture has allowed more advanced performance monitoring activities to be considered. Two of these developments are referred to as adaptive monitoring and automatic incident reporting (AIR).

Adaptive monitoring allows a monitoring schedule (a test) or a routine processing operation to be modified automatically in the light of new information. An example of this would be the automatic detection of an increase in average network delay at a certain time of day. This indicates a need for more detailed measurement of other metrics of interest. An additional test on the suspect route would then be automatically initiated. This avoids a need to continually test all the metrics of interest and minimises the test traffic in the network.

AIR is a mechanism to allow low-level test data to be automatically processed into a higher level of useful information for network operators or application-layer processes such as the DPS. In many cases, the users of a network are mainly concerned with the current and future well being of the network. This can often be deduced by an intelligent interpretation of packet or cell delay and loss characteristics. The AIR system uses a combination of techniques, including statistical matching to identify network routes and times when unusual characteristics of the network exist. The power of AIR is that it highlights these effects automatically, saving the human user the time-consuming task of manually inspecting the performance data generated by the monitoring system, and saving programmers the task of building expensive data filters into application-layer processes using the data.

9. Discussion

The work outlined here has all been demonstrated on hosts within BTL, much of it concurrently. The final deliverable for the current URI contract is an integrated deployment on LEANET [29] during the summer of 1999. This will be followed by further collaborative work on the outstanding research issues. Initial deployment of active service functionality in public networks is expected before 2002 with commercial services following soon after. The first deployment is expected to be of simple proxylets that cannot invoke further proxylets (i.e. only one per service) and operate exclusively in the application layer. The system will be a fully developed, high performance version of the

current ALAN, based around HTTP. It is anticipated that by around 2006 there should be complex interacting proxylets and a low-level VM for transport layer services. Information interoperability will be based on HTTP-NG, and the universal resource name (URN) scheme. There should also be a complete portfolio of information-layer services, such as proximity detection, name resolution and directories. By 2010 it is possible that active node hardware will be flexible too, and certain that it will be a tighter integration of routeing, storage and processing capability than currently available. Smart information able to advertise its presence on arrival at a node, and able to respond to service availability responses to configure itself for the local environment, will also be available by this date.

10. Conclusions

This paper has presented an outline design for an active information network environment, and discussed some of the key management, monitoring and control functions developed to support the environment. The first integrated demonstration of the environment is expected in August 1999 on the LEANET infrastructure. Key issues for further research have been outlined. Further collaboration with the academic authors is planned in order to carry out the research.

References

- Marshall I W and Bagley M: 'The information services supermarket — an information network prototype', *BT Technol J*, 13, No 2, pp 132—142 (April 1995).
- McKee P F, Marshall I W and Henning I D: 'Research directions in distributed systems', *BT Technol J*, 17, No 2, pp 137—144 (April 1999).
- Tennenhouse D and Wetherall D: 'Towards an active network architecture', *Computer Communication Review*, 26, No 2 (1996).
- Wakeman I et al: 'Designing a programming language for active networks', *HIPPARCH '98* (June 1998).
- Alexander D S, Shaw M, Nettles S M and Smith J M: 'Active bridging', *Computer Communication Review*, 27, No 4, pp 101—111 (1997).
- Lazar A: 'Programming telecommunication networks', *IEEE Network*, pp 2—12 (October 1997).
- Rooney S et al: 'Tempest: A framework for safe programmable networks', *IEEE Comms*, pp 42—53 (October 1998).
- Biswas J et al: 'The IEEE P1520 standards initiative for programmable interfaces', *IEEE Comms*, pp 64—72 (October 1998).
- Alexander D S et al: 'A secure active network environment architecture', *IEEE Network Magazine* (1998).
- Fry M and Ghosh A: 'Application layer active networking', *Fourth International Workshop on High Performance Protocol Architectures (HIPPARCH '98)* (June 1998), (longer version to appear in *Computer Networks and ISDN Systems* (1999)).
- Amir E, McCanne S and Katz R: 'An active service framework and its application to real time multimedia transcoding', *Proc SIGCOMM '98*, pp 178—189 (September 1998).
- Parulkar G et al: 'Active network node project', Washington University St Louis (1998).
- 'Extensible Markup Language (XML)', W3C Recommendation (February 1998) — <http://www.XML.com/aXML/testaXML.htm>
- Cao P, Zhang J and Beach K: 'Active cache: caching dynamic contents (objects) on the Web', *Proc Middleware '98 (Ambleside)* (1998).
- Wessels D: 'Configuring hierarchical Squid caches', *AUUG'97*, Brisbane, Australia (1997).
- The Object Management Group: 'The Common Object Request Broker: Architecture and Specification', Version 2.0, OMG Headquarters, 492 Old Connecticut Path, Framington, MA 01701, USA (July 1995).
- Rogerson D: 'Inside COM — Microsoft's Component Object Model', Microsoft Press (1997).
- Pryce N and Crane S: 'Component interaction in concurrent and distributed systems', *Proc Fourth International Conference on Configurable Distributed Systems*, IEEE Computer Society, Annapolis, MD, USA (May 1998).
- Sloman M: 'Policy driven management for distributed systems', *Journal of Network and Systems Management*, 2, No 4, pp 333—360, Plenum Press (1994).
- Marriott D and Sloman M: 'Implementation of a Management Agent for Interpreting Obligation Policy', *Proc of the IEEE/IFIP Distributed Systems Operations and Management Workshop (DSOM '96)*, L'Aquila (Italy) (October 1996).
- Lupu E and Sloman M: 'A policy-based role object model', *Proceedings of the 1st IEEE Enterprise Distributed Object Computing Workshop (EDOC'97)*, Gold Coast, Australia, pp 36—47 (October 1997).
- Marriott D: 'Management policy for distributed systems', PhD Dissertation, Imperial College, Department of Computing, London, UK (July 1997).
- 'The Common Object Request Broker: architecture and specification', Object Management Group, OMG Doc 91.12.1 — <http://www.omg.org/> (December 1995).
- Microsoft Corp and Digital Equipment Corp: 'The Component Object Model Specification', Draft Version 0.9, Microsoft Developers Network CD-ROM (October 1995).
- Nahrstedt K and Smith J M: 'The QoS Broker', *IEEE Multimedia*, 2, No 1, pp 53—67 (Spring 1995).
- Waddington D G and Hutchison D: 'A general model for QoS adaptation', *Proc of the Sixth International Workshop on Quality of Service (IWQoS'98)*, Napa, California, USA (May 1998).
- Phillips I W, Parish D J and Rodgers T: 'SMDS network performance measurement', *IEE Colloquium on: 'Practical experience with SMDS'*, IEE, pp 7.1—7.4, London (October 1995).
- Bashir O, Phillips I W, Parish D J, Adams J L and Spencer T: 'The management and processing of network performance information', *BT Technol J*, 16, No 4, pp 203—212 (October 1998).
- Gibbins C et al: 'Broadband Multicast on BT's Futures Testbed', *British Telecommunications Eng J*, 17, No 3, pp 139—149 (October 1998).



Ian Marshall leads a number of BT research projects in the area of Internet middleware, including the ALPINE project. He has been working in this area since 1992. Previously he worked in optical networks, broadband networks and network strategy.

He is a fellow of the Institute of Physics, a Chartered Engineer, and a Member of IEEE and ACM. He serves on several institute committees, and on EPSRC and European advisory panels. He is also an industry liaison panel member at several UK universities.



David Hutchison is Professor of Computing at Lancaster University, where he has worked for the past fifteen years.

He has published numerous papers, and edited several books, on services and support mechanisms for distributed multimedia applications.

He serves on advisory and assessment panels for the EPSRC and the European Commission, and is a technical expert adviser at Oftel.



Jon Crowcroft is a professor of networked systems in the Department of Computer Science, University College London, where he is responsible for a number of European and US funded research projects in multimedia communications. He has been working in these areas for over 18 years.

He is a member of the ACM, the British Computer Society and a Fellow of the IEE and a senior member of the IEEE. He is a member of the IAB, general chair for the ACM SIGCOMM, and on the editorial team for the ACM/IEEE Transactions on Networks.



David J Parish holds BSc and PhD degrees from the University of Liverpool. He has worked as a Scientific Officer at the UKAEA Culham Laboratory and as a Demonstrator in the Electrical Engineering Department at Liverpool University. From 1983 he has held the position of Lecturer and later Senior Lecturer in the Department of Electronic and Electrical Engineering at Loughborough University. His research interests concern the management, operation, monitoring and application of high performance networks. Specifically, he leads Loughborough's input to the BT-funded research programme into

the management of high-speed networks using SuperJanet.



Professor Michael Fry is the Dean of the Faculty of Mathematical and Computing Sciences at the University of Technology, Sydney (UTS). He has a BA from Cambridge University, an MSc from Imperial College London, and a PhD from the University of Sydney. He has been with UTS for 18 years. Prior to that he worked in the IT industry developing real-time systems.

He currently leads research projects investigating quality of service for multimedia delivery over the Internet, adaptive communication systems for mobile com-

puters, and infrastructure for value-added communication services.



Iain Phillips received his BSc and PhD degrees in Computer Science from the University of Manchester with his PhD research related to Load Balancing on Massively Parallel Computers. From 1992 he has worked as a Research Assistant in the High Speed Networks Group on work related to broadband network performance monitoring, teleconferencing over ATM, video error concealment, ATM-based surveillance systems, and systems integration. This work has been funded by EPSRC, RACE (Europe) and BT.

Specifically, his current research is on the performance monitoring and measurement of high-performance networks under the BT University Research Initiative (URI) using SuperJanet. He is a Graduate Student Member of the British Computer Society.



Atanu Ghosh received the BSc (Eng) from Queen Mary College, University of London in 1983, and the MSc in Data Communication Networks and Distributed Systems from University College, London in 1991.

He currently works at the School of Computing Sciences, University of Technology, Sydney, as a Senior Research Fellow.



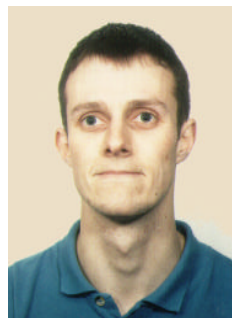
Nat Pryce currently works in the Distributed Software Engineering group of the Department of Computing, Imperial College, on the BT-funded Management of Multiservice Networks project, performing research into how to build, configure and manage multimedia services provided on broadband, multiservice networks.

This work has resulted in Regent, a set of languages, tools and object-oriented frameworks for building configurable distributed systems.



Professor Morris Sloman has been in the Department of Computing, Imperial College since 1976. He has managed many research projects funded by the EPSRC, EU and various industries on the management, security and design of distributed systems, multimedia systems and mobility.

He is chairman of the EPSRC Multimedia and Network Applications Funding Committee and is program co-chair of the 1999 IEEE/IFIP Integrated Management Symposium (IM'99).



Daniel Waddington is currently a final year PhD student working as a Research Assistant in the Distributed Multimedia Research Group at Lancaster University. For the last four years he has been involved in a five year BT University Research Initiative (BT-URI) project, investigating the management of multiservice networks.

His research interests include middleware support for distributed multimedia applications over broadband networks and operating system services for end-system resource management.